

Taller Teórico–Práctico

Proyecto Full-Stack: Frontend con Vue 3 y Backend con Node.js + Express

Programa: Desarrollo de Software

Autor: Prof. Uriel Castañeda Sierra

Introducción

En este taller el estudiante desarrollará un primer proyecto Full-Stack funcional, integrando un Frontend en Vue 3 con un Backend en Node.js y Express. El ejercicio está diseñado de forma evolutiva: primero se consumen datos simples, luego se organiza un backend propio y, finalmente, se incorporan recursos estáticos (imágenes). No se abordan autenticación ni patrones avanzados para priorizar la comprensión de los fundamentos.

Objetivos del taller

- Comprender la comunicación Frontend ↔ Backend mediante HTTP.
- Construir una API REST básica con Express.
- Consumir datos desde Vue usando fetch y manejar errores básicos.
- Comprender la diferencia entre URLs de datos y URLs de recursos estáticos.
- Incorporar imágenes desde un backend local.

PARTE 1 – BACKEND

1. Creación del proyecto Backend

1) Crear carpeta del proyecto y entrar en ella:

```
mkdir backend-api  
cd backend-api
```

2) Inicializar Node.js:

```
npm init -y
```

3) Instalar dependencias:

```
npm install express cors
```

```
npm install --save-dev nodemon
```

2. Estructura de carpetas del Backend

```
backend-api/  
├─ index.js  
├─ package.json  
└─ src/  
    ├─ app.js  
    └─ routes/  
        └─ productos.routes.js
```

3. Creación del servidor Express (src/app.js)

```
import express from 'express';  
  
import cors from 'cors';  
  
import productosRoutes from '../routes/productos.routes.js';  
  
const app = express();  
  
app.use(cors());  
app.use(express.json());  
  
app.use('/api/productos', productosRoutes);
```

```
export default app;
```

Explicación: express() crea el servidor; cors() permite que el frontend acceda; el prefijo /api/productos agrupa las rutas del recurso productos.

4. Endpoint de productos (src/routes/productos.routes.js)

```
import { Router } from 'express';

const router = Router();

router.get('/', (req, res) => {
  res.json([
    { nombre: 'Revitalift', foto: 'revitalift.avif', votacion: 120 },
    { nombre: 'Elseve', foto: 'elseve.jpg', votacion: 98 },
    { nombre: 'Infallible', foto: 'infallible.jfif', votacion: 150 },
    { nombre: 'Elvive', foto: 'elvive.avif', votacion: 87 },
    { nombre: 'True Match', foto: 'truematch.webp', votacion: 110 },
    { nombre: 'Preference', foto: 'preference.jpg', votacion: 65 },
    { nombre: 'Men Expert', foto: 'menexpert.jpg', votacion: 45 },
    { nombre: 'Age Perfect', foto: 'ageperfect.webp', votacion: 72 },
    { nombre: 'Casting Crème', foto: 'castingcreme.webp', votacion: 90 },
    { nombre: 'Studio Line', foto: 'studioline.jpg', votacion: 30 }
  ]);
});

export default router;
```

5. Arranque del servidor (index.js)

```
import app from './src/app.js';

const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
  console.log(`Servidor escuchando en http://localhost:${PORT}`);
});
```

Ejecución en desarrollo (reinicio automático con nodemon):

```
npm run dev
```

Ejecución normal:

```
npm start
```

Diferencia: npm run dev usa nodemon (reinicia al guardar); npm start usa node (arranque simple).

PARTE 2 – FRONTEND

6. Bloque único de variables (App.vue)

El bloque permite alternar entre API local y pública sin cambiar el resto del componente.

```
// BACK LOCAL (nuestra API):

const API_BASE = 'http://localhost:3000';

const API_PATH = '/api/productos';


// ----- API PÚBLICA -----

// const API_BASE = 'https://dummyjson.com';

// const API_PATH = '/products';
```

7. Fetch y manejo básico de error (App.vue)

```
<script setup>

import { ref, onMounted } from 'vue';

const productos = ref([]);
const error = ref(null);

onMounted(async () => {
  try {
    const res = await fetch(`${API_BASE}${API_PATH}`);
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    productos.value = await res.json();
  } catch (err) {
    error.value = `No se pudo cargar la información (${err.message})`;
    productos.value = [];
  }
});
</script>
```

8. Renderizado inicial (App.vue)

```
<template>

  <main>

    <h1>Aplicación ejemplo FRONT-END Vue</h1>

    <p v-if="error" style="color:red; font-weight:bold;">⚠ {{ error }}</p>

    <ul v-else>

      <li v-for="p in productos" :key="p.nombre">
```

```
        {{ p.nombre }} - {{ p.foto }} - {{ p.votacion }}  
    </li>  
</ul>  
</main>  
</template>
```

PARTE 3 – INCORPORACIÓN DE IMÁGENES (etapa final)

Propósito: mostrar cómo el backend sirve imágenes estáticas y el frontend construye la URL sin alterar la estructura de los datos.

9. Carpeta pública en el Backend

```
backend-api/  
└─ public/  
    └─ imagenes/  
        └─ productos/  
            ├── revitalift.avif  
            ├── elseve.jpg  
            └─ ...
```

10. Exponer estáticos (src/app.js)

```
import path from 'path';  
import { fileURLToPath } from 'url';  
  
const __filename = fileURLToPath(import.meta.url);  
const __dirname = path.dirname(__filename);  
  
app.use(express.static(path.resolve(__dirname, '../public')));
```

11. Mostrar la primera imagen debajo del listado (App.vue)

```
<script setup>
import { ref, computed, onMounted } from 'vue';

const API_BASE = 'http://localhost:3000';
const API_PATH = '/api/productos';
const productos = ref([]);
const primerProducto = computed(() => productos.value[0] ?? null);

function urlFoto(p) {
  if (!p?.foto) return '';
  if (/^https?:\/\//i.test(p.foto)) return p.foto; // si viene de API pública
  return `${API_BASE}/imagenes/productos/${p.foto}`; // si es local
}
</script>

<template>
  <!-- listado anterior -->
  <hr />
  <section v-if="primerProducto">
    <p>
      <strong>Nombre:</strong> {{ primerProducto.nombre }}<br />
      <strong>Foto (campo):</strong> {{ primerProducto.foto }}<br />
      <strong>Votación:</strong> {{ primerProducto.votacion }}
    </p>
    <img alt="urlFoto(primerProducto)" data-bbox="111 555 372 572" />
  </section>
</template>
```

12. Verificación final

- Probar en navegador: <http://localhost:3000/api/productos> (JSON)
- Probar una imagen directa: <http://localhost:3000/imagenes/productos/elseve.jpg>
- Ver en el Frontend: listado de productos y, bajo una línea <hr>, la imagen del primer producto.